

SVKM'S NMIMS
School of Technology Management & Engineering
Department of Computer Engineering

Image and Video Processing

Project Report

iHVAC: Intelligent HVAC Occupancy Mapping Room Automation

Submitted By:

Anupam Kanoongo - 70022400311(F044)
Arjun Borkar - 70022400057(F005)
Asmit Baldi - 70022400309(F042)
B.Tech CE Section A

Submitted To:

Dr. Raj Gaurav Mishra
Associate Professor

Abstract

Lighting and air conditioning in offices and classrooms often run on fixed schedules or manual settings, wasting energy in empty rooms. This project presents iHVAC, a camera-based system that estimates room occupancy from video and uses it to control lights and an air conditioner (AC). Frames from a fixed room camera are processed by a YOLO person detector with ByteTrack tracking. Detected bounding boxes are overlapped with a perspective-shaped grid of 8 by 6 zones, and each zone touched by a person is marked occupied. Lights change only after 10 stable seconds to prevent flicker, while AC settings follow the visible headcount through comfort profiles. A Tkinter dashboard displays occupancy maps, trends and replay. Techniques include frame acquisition, deep-learning detection, multi-object tracking, geometric grid mapping, polygon overlap analysis, temporal filtering and overlay rendering. In a logged live session of about four minutes (355 snapshots), the visible count ranged from 0 to 6, mean detector confidence was 0.77, and AC states matched the Balanced profile rules. Lit zones did not track headcount (one person covered about 40 of 48 zones on average), and no manually labelled evaluation has yet been done.

1. Introduction

1.1 Background of the Problem

Buildings use a large share of their electricity for lighting and space conditioning. In most rooms these systems are controlled by wall switches, fixed timers or a single thermostat setting. They keep running when the room is empty, and they respond slowly when it fills up. Common occupancy sensors have limits. Passive infrared (PIR) sensors detect movement, so they often switch lights off for people who sit still. CO2 sensors respond slowly to changes in headcount. Door counters record how many people enter or leave, but not where they are in the room.

1.2 Why the Problem Is Relevant

Energy spent on empty or lightly used rooms is wasted, and the same wasted energy adds to running costs and emissions. Comfort also suffers when a room is cooled to one temperature regardless of how many people are inside. A control system that knows how many people are present, and roughly where, can switch equipment only when it is needed and set temperatures to suit the actual use of the room.

1.3 Role of Image and Video Processing

A camera already watching a room carries spatial information that PIR and CO2 sensors do not provide. Video processing turns frames into this information. Object detection finds people in each frame. Tracking links detections over time so that the count and the positions remain stable. Geometric mapping places each person in a defined zone of the room. The output of these steps can drive light and AC decisions.

1.4 Motivation for Selecting the Project

The project combines a practical building problem with the main stages of a video processing pipeline: acquisition, detection, tracking, spatial analysis and display. It also needs careful engineering around

stable decisions, since a control system that flickers or toggles equipment on every noisy frame is not usable. This made it a useful test of both the image processing ideas and their behaviour in a real setting.

2. Problem Statement and Objectives

Problem statement: Given a fixed room camera, estimate in near real time how many people are visible in the room and where they are, and convert this into light and AC control decisions that are stable and do not run equipment unnecessarily.

Objectives of this project:

1. Detect people in each video frame using a pretrained YOLO model restricted to the person class.
2. Track detected people across frames with ByteTrack so that per-frame detections are stable.
3. Map each detection onto a configurable perspective grid of room zones (default 8 by 6) and mark the zones that are occupied.
4. Convert zone occupancy into debounced light states and headcount-based AC settings using selectable comfort profiles.
5. Present occupancy, zone maps, trends and system health in a dashboard, and log readings for later analysis.

3. Scope of the Project

What the project covers:

- Live input from a camera (by index or stream URL) or from a recorded video file.
- Person detection and tracking, perspective zone mapping and occupancy calculation.
- Light and AC control logic, including a 10-second light settling period and four comfort profiles (Balanced, Focus, Meeting, Eco).
- A Tkinter dashboard with occupancy and light map layers, a trend slider with replay, and room setup controls.
- CSV and JSON logging of readings, zone snapshots and device states.

What the project does not cover:

- Identification of individuals. The system does not use face recognition and does not store images unless an output video path is given.
- Measured energy savings. The repository states that iHVAC does not calculate savings; power and energy values must come from a real meter.
- Full camera calibration. The zone grid is parametric (columns, rows, tilt) and is not calibrated against real-world measurements with a homography.
- Multi-room or multi-camera operation.
- Classical pixel-level processing such as thresholding, morphology, optical flow or background subtraction (see Section 6).

Type of images and videos: Fixed-viewpoint indoor RGB video from a camera tilted downward at about 30 degrees.

Application domain: Smart buildings, building energy management and Internet-of-Things (IoT) control of HVAC and lighting.

4. Dataset / Input Description

Source of input: A live room camera (default source index 0 in config.json) or a video file supplied with the --source option. No public dataset was used to train the detector. The project uses pretrained Ultralytics YOLO checkpoints bundled in the repository: yolo26n.pt (about 5.5 MB), yolo11n.pt (about 5.6 MB) and yolo11m.pt (about 40.7 MB). The default is yolo26n.pt.

Dataset name: Not applicable for training. The detector's pretrained weights are trained on the COCO dataset.

Number of images or videos: The repository holds one logged live session of about four minutes, which produced 355 zone snapshots, and a longer event log covering 14 September to 9 October 2026.

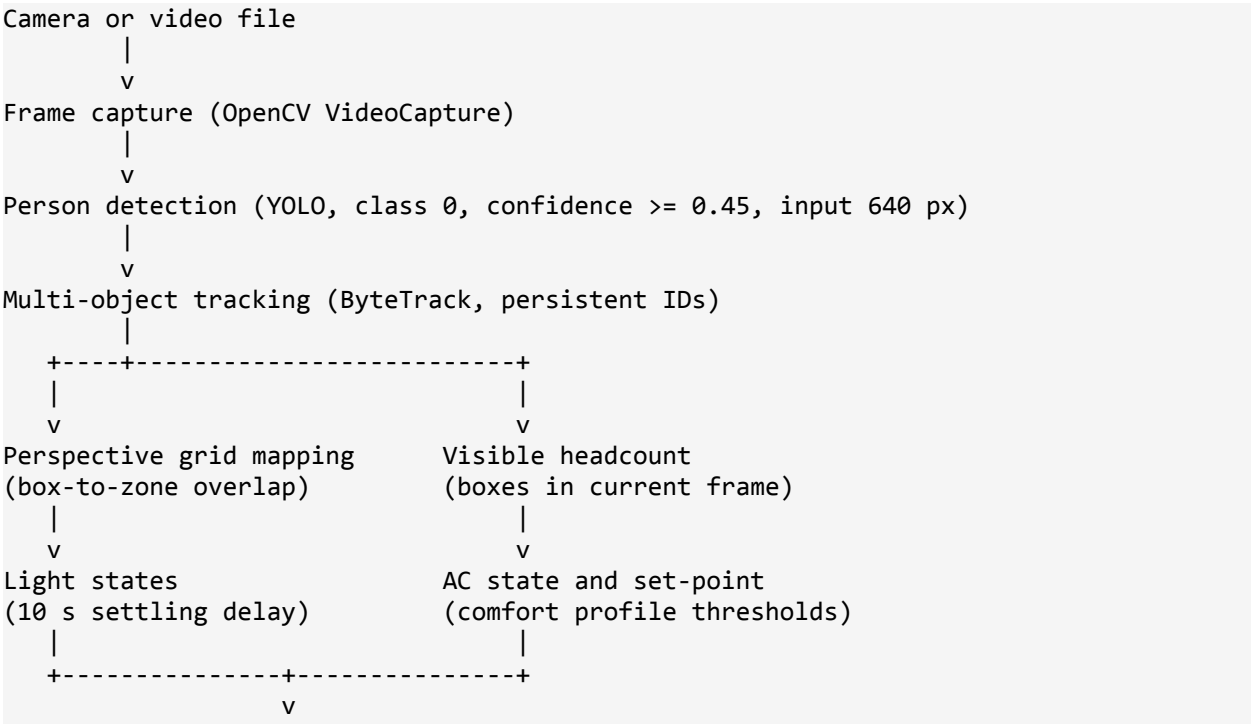
Image resolution and format: Frames are read as BGR arrays by OpenCV at the camera's native size, and the grid adapts to the frame size. Optional recordings are written as MP4 (mp4v codec) at 30 frames per second.

Video resolution and duration: The logged live session lasted from 15:39:59 to 15:43:49 on 9 October 2026.

Sample input frames:

5. Methodology / Proposed Approach

The system processes each video frame through the pipeline shown below.



```
Optional relay outputs (Arduino/Firmata), JSON state,
CSV logs, Tkinter dashboard, overlay video
```

The main loop can be summarised by the following pseudocode:

```
for each frame F from the video source:
    people, mean_conf = track(YOLO, F, class = person, conf >= 0.45)
    count = number of people
    occupied = { zone c : area(box b intersect polygon c) > 1 px^2,
                 for every box b and zone c }
    for each zone c:
        if desired state (c in occupied) differs from stored state:
            commit the new state only if it has persisted for 10 s
    if count < ac_on_at:            AC = OFF, set-point = 26 C
    else if count >= busy_at:       AC = ON,  set-point = 21 C
    else:                           AC = ON,  set-point = 23 C
    write AC and light outputs (if a board is connected)
    update state JSON, zone history CSV and event CSV
    draw zone overlay and person boxes on F
```

Perspective zone grid. The frame is divided into rows and columns with a perspective shape. Row boundaries are placed at heights proportional to (i / rows) raised to an exponent derived from the tilt angle, so rows near the top of the frame, which are farther from the camera, are thinner. Columns converge toward the centre of the frame at the top, with a strength set by the tilt angle. At the default tilt of 30 degrees the exponent is about 1.73 and the convergence weight is about 0.61. This is a parametric approximation rather than a calibrated homography.

Occupancy rule. A zone is occupied when a person's bounding box overlaps it by more than 1 square pixel. Any touched zone counts, so a person who stands across a zone boundary occupies more than one zone.

Light stability. A light changes state only after the desired state has held for 10 seconds. A change that reverts within that time is discarded.

6. Techniques / Algorithms Used

The table below lists the image and video processing techniques named in the course outline and states whether each is used in iHVAC.

Technique	Used?	How it is used or why it is not
Grayscale conversion	No	Detection runs on colour frames, which the YOLO model expects.
Histogram equalization	No	The detector is robust to the lighting in the test setting, so contrast enhancement was not added.

Technique	Used?	How it is used or why it is not
Thresholding	Partly	Detector confidence is thresholded at 0.45. No pixel-level threshold is applied.
Filtering	No spatial filter	Overlay drawing uses alpha blending (cv2.addWeighted) only.
Edge detection	No	Not needed; the detector learns its own features.
Morphological operations	No	Not used.
DFT / DCT	No	Not used.
Image segmentation	Partly	The frame is partitioned into zones. This is a spatial partition, not semantic or instance segmentation.
Feature extraction	Yes	Deep features are learned inside the YOLO network. Box centroids and box corners are extracted as geometric features.
Frame extraction	Yes	Frames are read one at a time with cv2.VideoCapture.
Background subtraction	No	A detector was chosen because background models fail when lighting changes and when people stay still.
Optical flow	No	Tracking by ByteTrack provides motion association without a dense flow field.
Motion detection	Implicit	Motion is inferred through tracking; no separate motion detector is used.
Object tracking	Yes	ByteTrack, enabled with persist=True, keeps identities across frames.
Frame differencing	No	Not used.

Object detection. A YOLO convolutional network detects objects in each frame. The system keeps only the person class (class 0), uses an input size of 640 pixels and accepts detections with confidence of at least 0.45.

Multi-object tracking. ByteTrack associates detections across frames using their boxes and confidence scores. The tracker is enabled with persistent state, so identities survive from one frame to the next.

Geometric overlap. Each person's bounding box is tested against each zone polygon with cv2.intersectConvexConvex. Both shapes are convex quadrilaterals, so this test is exact.

Temporal filtering. The 10-second settling rule applied to lights is a simple temporal filter. It removes short changes in occupancy so that lights do not flicker.

7. Implementation

Programming language: Python 3.14.

Software and tools: Visual Studio Code or a terminal for development; the Tkinter GUI toolkit for the dashboard; a JSON file (config.json) for hardware and comfort settings.

Libraries and frameworks:

- OpenCV (opencv-python, version 4.7 or later) for video capture, overlay drawing and polygon intersection.
- Ultralytics (version 8.4 or later) for YOLO inference and ByteTrack tracking. PyTorch is installed as the deep learning backend of Ultralytics.
- NumPy for zone geometry and array handling.
- pyFirmata (version 1.1.0) for optional communication with an Arduino running the Firmata firmware.
- Tkinter, json, csv and dataclasses from the Python standard library.

Hardware: Any camera supported by OpenCV, mounted to view the room from above at a downward tilt of about 30 degrees. An optional Arduino board with relays drives the AC on the pin set by AC_Pin and up to 48 light outputs set by Light_Pins. Inference runs on the CPU by default; a GPU is optional.

Important implementation steps:

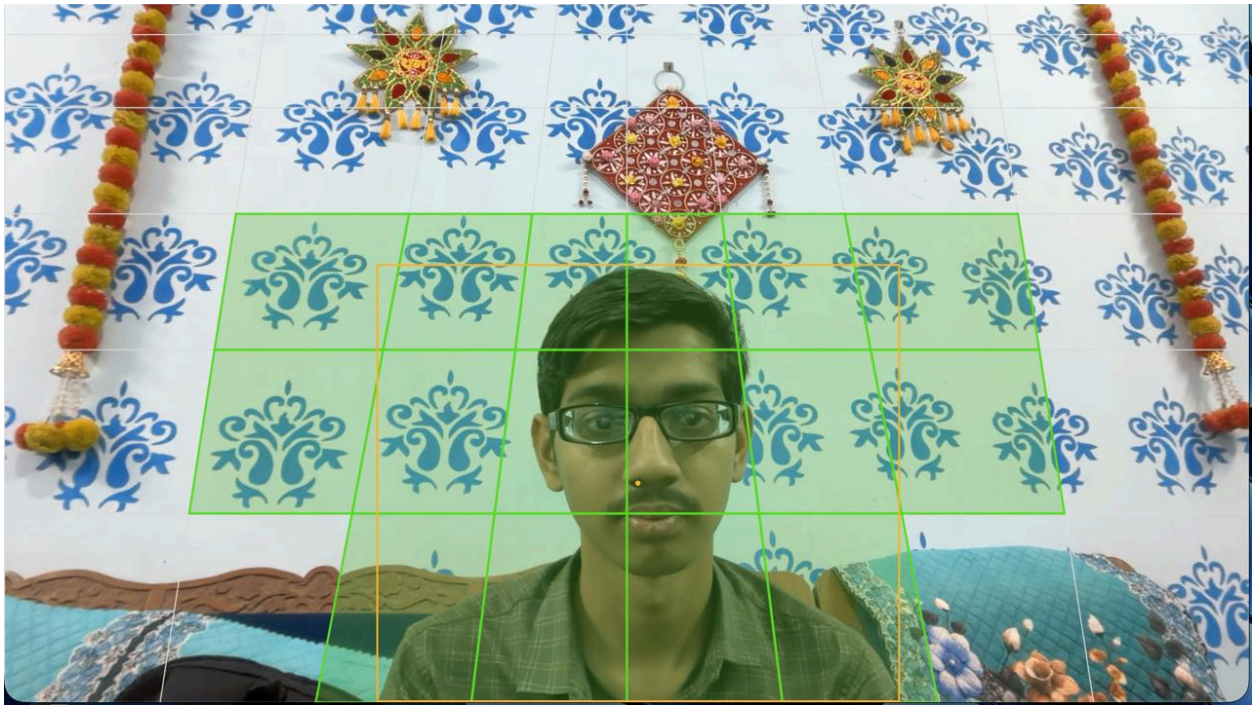
1. Read config.json and command-line options (source, model, tracker, confidence, output path, save interval).
2. Open the video source and load the YOLO model.
3. For each frame, run tracking and extract boxes and centroids for the person class.
4. Compute occupied zones from box-to-polygon overlap.
5. Update light states with the 10-second settling rule and write the outputs.
6. Select AC state and set-point from the visible headcount and the active comfort profile.
7. Write the JSON state, the zone history CSV and the event CSV, and draw the overlay on the frame.
8. Re-read config.json once per second. A change to grid size, tilt or light pins rebuilds the grid and the light controller without restarting the process.

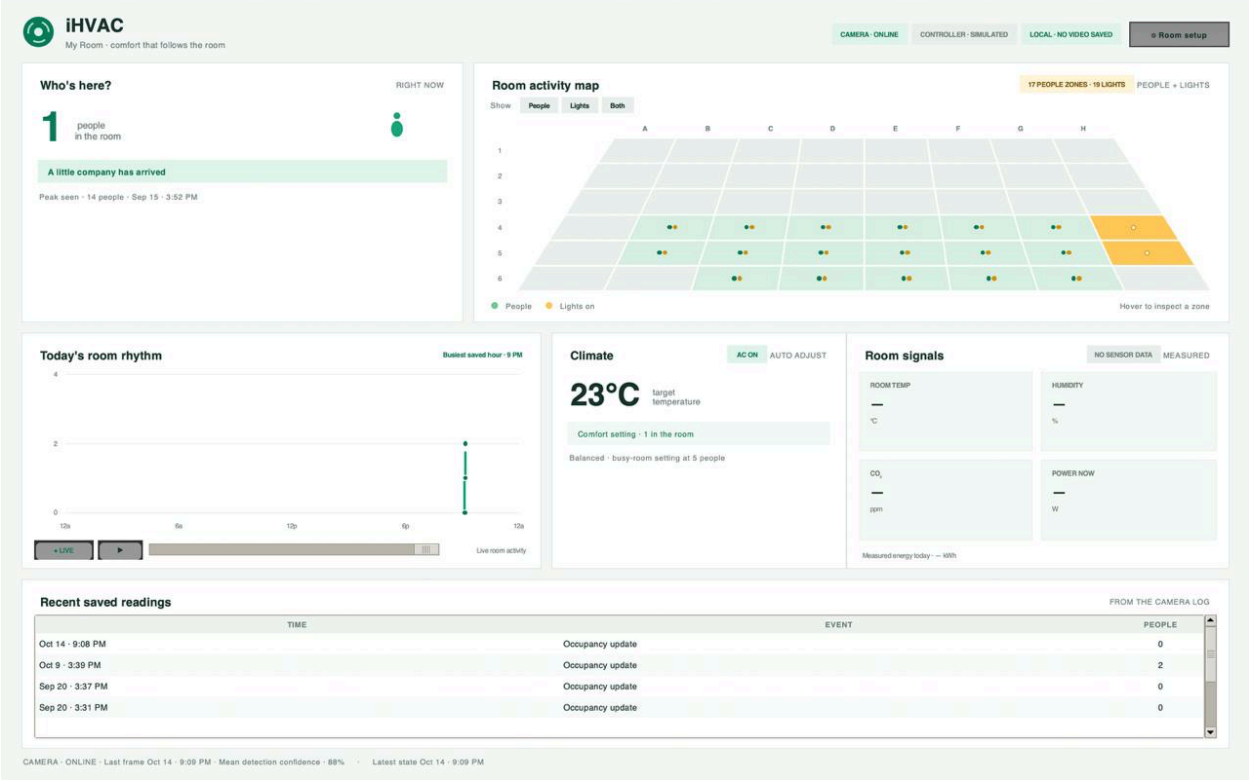
The central overlap test is short enough to quote:

```
for cell, polygon in cell_polygons.items():
    intersection_area, _ = cv2.intersectConvexConvex(person_polygon, polygon)
    if intersection_area > 1.0:
        occupied.add(cell)
```

8. Results and Discussion

8.1 Input versus Output





The overlay marks each detected person with a box and centre point. Occupied zones are filled in green, and the zone outlines show the perspective grid. The dashboard shows the same zone state as a separate map with People, Lights and Both views.

8.2 Logged Live Session: Occupancy Distribution

The room history log records 355 snapshots between 15:39:59 and 15:43:49 on 9 October 2026. Each snapshot records the visible count, the occupied zones, the light states and the AC state.

Visible people	Snapshots	Share of snapshots	AC state (Balanced profile)
0	24	6.8%	OFF, 26 C
1	70	19.7%	ON, 23 C
2	83	23.4%	ON, 23 C
3	75	21.1%	ON, 23 C
4	57	16.1%	ON, 23 C
5	39	11.0%	ON, 21 C
6	7	2.0%	ON, 21 C

Across the session, the AC was OFF in 24 snapshots (6.8%), at 23 C in 285 snapshots (80.3%) and at 21 C in 46 snapshots (13.0%). These counts match the thresholds of the Balanced profile exactly: one or more people turn the AC on, and five or more select the lower set-point.

8.3 Detector Confidence

Detector confidence was recorded in 331 snapshots, the other 24 being empty-room snapshots. The mean confidence was 0.769, with a minimum of 0.452 and a maximum of 0.940. The minimum sits just above the 0.45 threshold, which shows that some accepted detections are borderline.

8.4 Lit Zones versus Headcount

Visible people	Snapshots	Mean lit zones (of 48)	Minimum	Maximum
0	24	0.0	0	0
1	70	39.8	12	48
2	83	39.6	12	48
3	75	34.8	15	48
4	57	27.3	14	46
5	39	26.0	18	46
6	7	21.7	21	22

The number of occupied zones does not increase with headcount. A single visible person occupies about 40 of the 48 zones on average, and at times all 48 zones. Two people occupy about as many zones as one. Because each person's box is large relative to the zone size, and any overlap above 1 square pixel counts, the occupancy map marks far more area than the people occupy. Across the session, the average number of lit light zones was 34.2 of 48. This means the lighting logic behaves more like a presence map than a location map. The AC logic is not affected, because it uses the headcount directly rather than the zone count.

Two changes would address this: a minimum overlap fraction, such as requiring 25 percent of a zone to be covered, and using only the lower part of each box (the feet or lower body) to decide the zone.

8.5 Quantitative Measures Still to Be Collected

The logs describe the behaviour of the system but do not measure its accuracy, because no frames were manually labelled. The table below defines the measures planned for the evaluation.

Measure	Definition	Status
Count accuracy	Mean absolute error between the predicted and true headcount per frame	Not yet measured

Measure	Definition	Status
Detection precision and recall	Matched boxes at IoU of 0.5 against manual annotations	Not yet measured
Zone accuracy	Share of frames where predicted occupied zones match the annotated zones	Not yet measured
Light flicker rate	Number of light state changes per hour, with and without the 10-second rule	Not yet measured
Processing rate	Frames per second on the target computer, with yolo26n and yolo11m	Not yet measured

9. Challenges / Limitations

Coarse zones and large boxes. As Section 8.4 shows, the zone map over-marks occupancy. This is the most visible limitation of the lighting logic.

Visible occupancy only. The count is the number of people detected in the current frame. People who are hidden behind furniture or other people are not counted. The README describes the output as visible occupancy for this reason.

Frame counts, not tracks. The count uses the number of boxes in each frame, not the number of unique track IDs. Tracking keeps the boxes consistent, but it does not remove the effect of a missed detection on the count.

No debounce for AC. The light outputs have a 10-second settling rule, but the AC state is computed from the current frame count and written immediately. A single frame in which a person is missed could switch the AC off or change the set-point. A similar settling rule should be added to the AC.

Perspective approximation. The zone geometry is not calibrated against measurements in the room, so the zone boundaries do not correspond exactly to physical positions in the room.

Detector confidence. Mean confidence was 0.77, and some accepted detections were close to the 0.45 threshold. False positives, such as mannequins or reflections, were not recorded.

Dataset limitations. No labelled ground truth exists in the repository, so the accuracy of counts and zones has not been measured. The only live data is a four-minute session. Lighting variation, camera angle and crowd density were not tested systematically.

Log consistency. The event log uses a column layout (track_id, entries_today, exits_today) that the current code no longer writes, which indicates that it was produced by an earlier version. The state file records a peak occupancy of 14 at 15:52:53 on 15 September 2026, which could not be reproduced from the history log, which does not cover that period. This value should be verified before it is reported.

Sensors and hardware. No environmental sensor file was present, so the temperature, humidity, CO2 and power readings are shown as unavailable. The controller was configured as not connected and no light pins were set, so the relay behaviour was not tested in the logged runs. The logs show calculated states only.

Computational limits. Processing rate was not measured. YOLO inference at 640 pixels on a CPU may not keep pace with a high frame rate camera, and the larger yolo11m model would be slower than yolo26n.

Privacy. The system stores counts, zone identifiers and device states. It does not store images unless an output path is given, which is an advantage for deployment in occupied rooms.

Failure cases. Failure cases were not systematically recorded. Based on the design, the following conditions are expected to degrade performance and should be confirmed with recorded clips: low light, strong backlight from a window, groups of people whose boxes merge, people at the frame edge who are partly out of view, and people seated behind desks where only the upper body is visible.

10. Conclusion

What was implemented. A complete occupancy-driven control pipeline: camera input, person detection, multi-object tracking, perspective zone mapping, debounced light control, headcount-based AC control with comfort profiles, a Tkinter dashboard and CSV and JSON logging.

Techniques used. YOLO deep-learning detection, ByteTrack tracking, geometric grid mapping, polygon overlap analysis, temporal settling and alpha-blended overlay rendering. Classical pixel-level methods such as thresholding, morphology, optical flow and background subtraction were deliberately not used.

What was achieved. The system runs end to end. In a logged live session, visible headcounts from 0 to 6 produced the AC states defined by the Balanced profile, and the dashboard reported occupancy, zone maps and device status. The data show that the zone-based light map over-marks occupancy, which is the main shortcoming to be corrected.

Learning. [Student reflection: describe what you learned about detection and tracking in practice, about the gap between a working demo and a stable control system, and about the value of measuring accuracy against ground truth.]

Future work. Annotate a test set and report precision, recall, count error and zone accuracy; add a minimum overlap fraction or foot-point rule for zones; add settling to the AC; calibrate the grid against measured room dimensions; and test the Arduino relay outputs on a real AC unit.

References

1. Bradski, G. (2000). The OpenCV Library. Dr. Dobb's Journal of Software Tools.
2. Gonzalez, R. C., & Woods, R. E. (2018). Digital Image Processing (4th ed.). Pearson.
3. Hartley, R., & Zisserman, A. (2004). Multiple View Geometry in Computer Vision (2nd ed.). Cambridge University Press.
4. Paszke, A., et al. (2019). PyTorch: An imperative style, high-performance deep learning library. Advances in Neural Information Processing Systems 32.

5. Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016). You only look once: Unified, real-time object detection. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 779-788.
6. Ultralytics. YOLO documentation. https://docs.ultralytics.com
7. Zhang, Y., Sun, P., Jiang, Y., et al. (2022). ByteTrack: Multi-object tracking by associating every detection box. Proceedings of the European Conference on Computer Vision.